



Google Summer of Code 2026

Native Streaming Transforms for the Apache Beam Python SDK

UnboundedSource Wrapper & Watch Transform

Applicant	Elia
Email	elialiulzy@gmail.com
GitHub	Eliaaazzz
Organization	The Apache Software Foundation (Apache Beam)
Mentors	Yi Hu (yhu@apache.org)
Project Size	~350 hours · Large

Contents

1	Proposal Summary	2
2	About Me	2
2.1	Prior Contributions to Apache Beam	2
2.2	Skills Alignment	3
2.3	Gaps and How I’m Addressing Them	3
3	Background	3
4	Design / Description of Work	3
4.1	Module Structure	3
4.2	UnboundedSource Wrapper (Essential)	4
4.2.1	Strategy	4
4.2.2	API Contract	4
4.2.3	Deduplication Contract	5
4.2.4	Checkpoint-Based Pause/Resume	5
4.2.5	Dynamic Splitting	6
4.2.6	Watermark, Checkpoint Finalization, and Failure Semantics	6
4.3	Watch Transform (Essential)	7
4.3.1	API Design	7
4.3.2	SDF Internals	7
4.3.3	Hash Stability: A Correctness Requirement	8
4.3.4	Memory Management and State Eviction	9
4.3.5	Termination Conditions	9
4.4	Comprehensive Test Suite (Essential)	9
4.5	User-Facing Documentation (Essential)	10
4.6	Optional / Stretch	10
5	Proof of Concept and Early Validation	10
6	Essential Deliverables	11
7	Results for the Apache Beam Community	11
8	Schedule (12 Weeks; May 25 – August 16, 2026)	11
9	Other Commitments	13
10	Community Engagement	13
11	Risks and Mitigations	14
12	References	14
A	UnboundedSource ABC Definitions	15
B	UnboundedSource SDF Pseudocode	16
C	Watch SDF Core Loop	17

1. Proposal Summary

This project aims to bring two native streaming primitives to the Python SDK: UnboundedSource as a source API for checkpointed custom streaming IO, and Watch as a transform for polling-based growth detection.

2. About Me

I am Elia, a final year Computer Science student at the University of Melbourne focusing on data infrastructure and backend systems. Previously, I worked at Aiwason as a software engineer. I also built a full-stack fitness application, Meritful (<https://aurafitness.org>) including its Java backend.

My passion for open-source development and interest in the mechanics of dataflow led me to Apache Beam, where I have been contributing to both the Java and Python SDKs.

2.1 Prior Contributions to Apache Beam

PR #37965 (under review):

Fixes a parallelization bug in the Python PortableRunner: `iobase.Read` uses a Splittable DoFn internally, but the portable runner's default optimization pipeline omitted the `expand_sdf` phase, causing runners such as Spark to execute Python reads (e.g., `ReadFromParquet`) on a single partition. Adds `expand_sdf` to the default optimization phases with test coverage.

PR #37278:

Fixed a grammar error in Beam's streaming configuration docs.

Smart Bucketing (PR #37565, #37532, #37945):

Designed and implemented a complete smart bucketing pipeline for Beam's Python ML inference across three PRs, minimizing padding waste for variable-length inputs:

- **#37565**: Implemented length-aware keying in `BatchElements` to minimize padding for variable-length workloads. Extended the public configuration API and added benchmarks covering skewed-distribution scenarios.
- **#37532** (under review): Adds `SortAndBatchElements`, a stateless bundle-local size-aware batching path for variable-length inputs. Benchmarked on heavy-tail Pareto distributions, showing a 75.5% reduction in padding ratio, 3.3× throughput improvement, and 69.7% reduction in E2E latency (p95).
- **#37945** (under review): Completes the integration by exposing `batch_length_fn` and `batch_bucket_boundaries` on all concrete `ModelHandler` subclasses (16 total) across PyTorch, HuggingFace, scikit-learn, TensorFlow, ONNX, XGBoost, TensorRT, vLLM, Vertex AI, and Gemini backends, with end-to-end and forwarding test coverage.

PR #37428:

Added content-aware dynamic batching to `RunInference` via an `element_size_fn` parameter. Previously, `RunInference` batched purely by element count, which worked poorly for NLP/LLM workloads where input sizes vary by orders of magnitude. The new parameter allows batching by actual content size. This work laid the groundwork for the Smart Bucketing effort above, which extends the approach from size-aware batch splitting to padding-minimized bucketed batching.

PR #37355:

Fixed `DoFnInvoker` cache collisions for generic types in `ByteBuddyDoFnInvokerFactory` by upgrading the cache key to include generic `TypeDescriptors`, which required reproducing the exact failure in core Java, and added a regression test.

PR #37299:

Fixed an `AttributeError` in `ExternalTransform.expand()` by replacing direct access to the `internal_type_hints` attribute with the public `get_type_hints()` accessor, with a regression test covering the cross-language and external-transform paths.

2.2 Skills Alignment

My merged Beam PRs cover both the Python and Java SDKs and include pytest-based tests, so I already have working knowledge of Beam’s testing style, CI workflow, and contribution process. I also built a PoC for this project, which involved reading the related Java code including `Watch.java`, `UnboundedSource.java`, `UnboundedSourceAsSDFWrapperFn` and implementing Python prototypes that exercise checkpoint, watermark, and SDF execution semantics.

My recent Beam contributions also touched stateful transforms, batching internals, ML-related APIs, and cross-language expansion. This gets me familiar with the codebase that this project will extend. I have also worked through the normal GitHub open-source process, including code review, follow-up iterations, and CI debugging.

2.3 Gaps and How I’m Addressing Them

Although I lack experience authoring a full SDF-based streaming IO connector for Beam, I have proactively bridged this knowledge gap. During my proposal preparation, I dissected `UnboundedSourceAsSDFWrapperFn` and `Watch.java`, and built local PoCs to firmly grasp and validate the underlying execution lifecycle.

3. Background

Beam’s Python SDK is increasingly popular, making robust native APIs crucial for developers. This proposal aims to port two essential streaming primitives natively to Python: `UnboundedSource` (for custom IOs) and `Watch` (for polling-based inputs), leveraging existing SDF primitives.

Native implementations are essential for developer convenience. Cross-language transforms add gRPC overhead and Java dependencies, while raw SDFs expose the steep learning curve of `RestrictionTracker`. Porting `UnboundedSource` and `Watch` directly in Python abstracts away these complexities and unlocks native streaming patterns for the Python ecosystem.

4. Design / Description of Work

4.1 Module Structure

The implementation touches the following modules in the Beam Python SDK.

Path	Purpose
<code>apache_beam/io/iobase.py</code>	Add <code>UnboundedSource</code> , <code>UnboundedReader</code> , <code>CheckpointMark</code> ABCs; extend <code>Read()</code> expansion to handle <code>UnboundedSource</code>
<code>apache_beam/io/watch.py</code>	New file: <code>Watch.growth_of()</code> , <code>WatchGrowthFn</code> , <code>GrowthState</code> , <code>TerminationConditions</code>
<code>apache_beam/io/fileio.py</code>	Stretch: refactor <code>fileio.MatchContinuously</code> to use <code>Watch</code>

Table 1: Target modules.

4.2 UnboundedSource Wrapper (Essential)

4.2.1 Strategy

I will implement `UnboundedSource` as a Composite Transform that expands into a Splittable DoFn, mirroring Java's `UnboundedSourceAsSDFWrapperFn`. The wrapper adapts the legacy `UnboundedReader` API into the SDF `RestrictionTracker` pattern so that users write simple Reader classes while the runner handles checkpointing, splitting, and watermark tracking.

4.2.2 API Contract

Abstract base classes in `apache_beam.io.iobase`, preserving Java semantics with Pythonic method names:

Class	Methods / Contract
<code>Read(source)</code>	Existing entry point; <code>expand()</code> extended with <code>isinstance(source, UnboundedSource)</code> check to dispatch to SDF wrapper
<code>CheckpointMark</code>	<code>finalize_checkpoint()</code> , <code>get_offset_limit()</code> (optional; if <code>offset_based_deduplication_supported()</code>)
<code>UnboundedReader</code>	<code>start()</code> , <code>advance()</code> , <code>get_current()</code> , <code>get_current_timestamp()</code> , <code>get_watermark()</code> , <code>get_checkpoint_mark()</code> , <code>get_current_record_id()</code> , <code>get_current_record_offset()</code> (optional; if <code>offset_based_deduplication_supported()</code>), <code>get_split_backlog_bytes()</code> (optional), <code>get_total_backlog_bytes()</code> (optional), <code>close()</code>
<code>UnboundedSource</code>	<code>split(desired_num_splits, options)</code> , <code>create_reader(options, checkpoint_mark)</code> , <code>get_checkpoint_mark_coder()</code> , <code>offset_based_deduplication_supported()</code> (optional), <code>requires_deduping()</code>

Table 2: `UnboundedSource` API contract.

Full ABC and restriction definitions are provided in [Appendix A](#).

Additional semantic guarantees mirrored from the Java contract:

- `start()` runs exactly once before any later read methods are used. That includes `advance()`, `get_current()`, and `get_current_timestamp()`.
- `advance()` returning `False` means “no record is currently available,” not “the source is permanently exhausted”. The wrapper must pause and resume later.
- `get_watermark()` is treated as monotonic from the wrapper’s perspective. If the reader regresses, the wrapper preserves the last emitted watermark.

`Read(source)` remains the canonical public entry point. The proposal extends the existing `isinstance` check in `Read.expand()` (`iobase.py`, around Line 937) so that passing an `UnboundedSource` dispatches to the new SDF wrapper, just as `BoundedSource` already dispatches to its bounded SDF path. This preserves the Pythonic user model (`Read(MyUnboundedSource())`) instead of introducing a parallel entry point. All new public APIs follow Beam’s Python SDK naming conventions (`snake_case` methods, Pythonic idioms).

`PipelineOptions` will be captured inside wrapper internals instead of relying on an implicit `process()` parameter path. The options-plumbing path will be serialization-safe.

4.2.3 Deduplication Contract

Dedup is opt-in. Note: `requires_deduping()` uses Pythonic naming; the Java equivalent is `requiresDeduping()`.

When `requires_deduping()` is `True`, the wrapper emits stable record IDs under the following contract: `get_current_record_id()` must return a byte string that is stable across checkpoint/restore cycles for the same logical record (same record, same ID, regardless of worker). IDs must also be unique within the source's output space (two distinct records must never produce the same ID). When `requires_deduping()` is `False`, `get_current_record_id()` will not be called.

This behavior will be validated during early implementation. The test will reconstruct a reader from the serialized `CheckpointMark` state and verify that the same logical record yields the same record ID after restore. The implementation will also check whether Python runner paths honor `ValueWithRecordId` for end-to-end deduplication instead of assuming support upfront. If native runner support is incomplete, that limitation will be noted and dedup correctness will be scoped to runners that honor `ValueWithRecordId`. A portable fallback (such as a stateful `DoFn`-based filter keyed on record ID) will only be pursued if its tradeoffs are acceptable to the community. Otherwise, the gap will be recorded as a known limitation in D4.

4.2.4 Checkpoint-Based Pause/Resume

The wrapper follows normal SDF execution rules. If the tracker rejects `try_claim(position)`, the current invocation must stop immediately.

When no data is available, the process loop returns `ProcessContinuation.resume(...)`, allowing the runner to resume later from checkpoint state instead of keeping a live reader across invocations.

Every `try_claim` call passes `None` as a pure gating check; the tracker only verifies that the restriction has not been marked done by a prior checkpoint. The element is read from the reader separately by `process()` after `try_claim` succeeds. This keeps the tracker's responsibility limited to liveness gating and avoids coupling it to element structure. To match the Java wrapper, `try_split()` returns `None` until the reader has been started.

To reduce the cost of repeated checkpoint/restore cycles, the wrapper caches the live reader instance across `process()` invocations within the same worker, mirroring the reader-caching pattern in Java's `UnboundedSourceAsSDFWrapperFn`. On resume, if a cached reader for the same source and checkpoint is available, the wrapper reuses it instead of calling `create_reader()` and `start()` from scratch. The cache is keyed by the structural value of the `(source, checkpoint)` pair, and invalidated whenever a checkpoint constructs a new restriction (e.g., after `try_split()`). This avoids redundant I/O setup costs, particularly for sources where reader initialization involves network connections or resource allocation.

Runner note: `ProcessContinuation.resume(delay)` is advisory, so correctness must not depend on exact resume timing. Runners may also checkpoint more aggressively than expected or lack full support for certain SDF features (e.g., self-checkpointing, dynamic splitting). The wrapper must tolerate all of this without breaking invariants; any runner-specific gaps found during testing will be recorded.

The tracker exposes a boolean `try_claim` gate and a checkpoint-based `try_split()`. The process loop drives the reader and decides what to yield. Full pseudocode for the tracker and process loop is provided in [Appendix B](#).

The PoC already validates this pattern: restriction creation, checkpoint round trip, watermark restore, deferred resumption, and end-to-end reader behaviors.

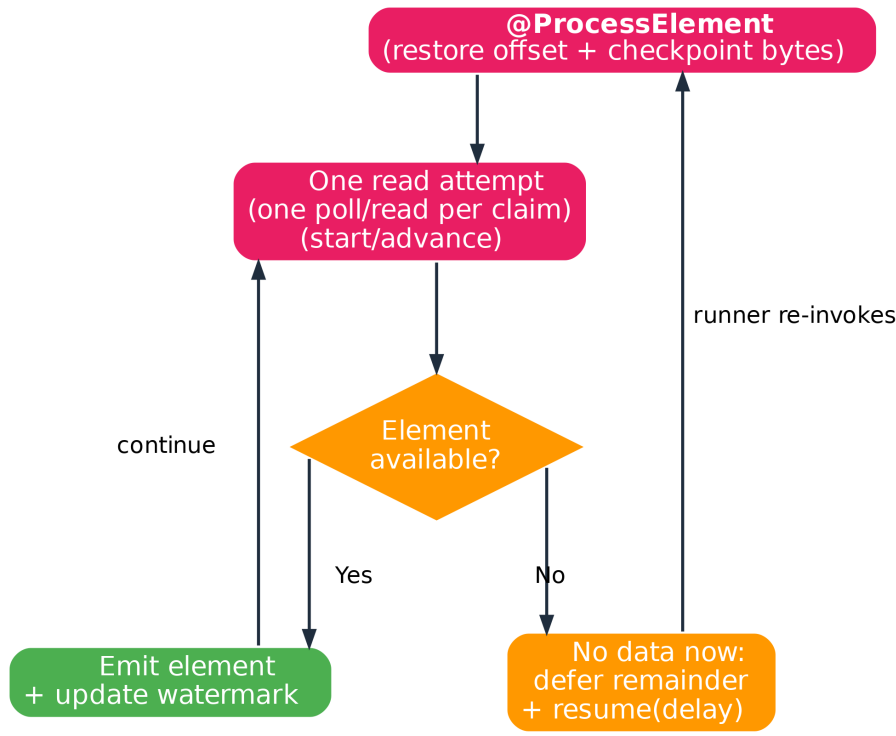


Figure 1: `UnboundedSource` wrapper: liveness-safe pause/resume (no busy spin).

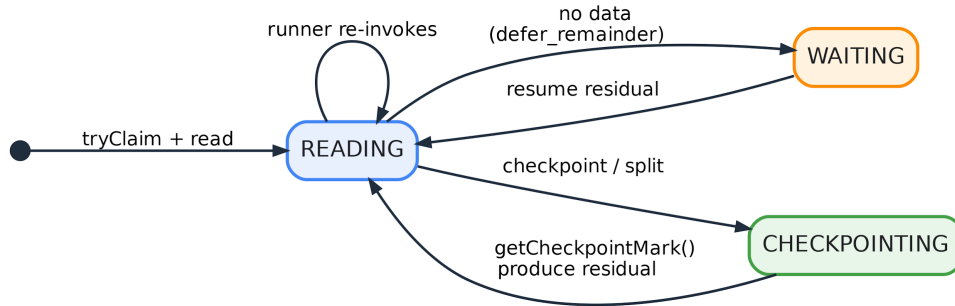


Figure 2: `_SDFUnboundedSourceRestrictionTracker` state machine (`READING` / `WAITING` / `CHECKPOINTING`).

4.2.5 Dynamic Splitting

Splitting is checkpoint-based, not fraction-based, because a live `UnboundedReader` has no meaningful fraction-position space. Before execution, splitting delegates to `source.split(...)`. Once a reader is active, any `try_split()` call constructs a residual from `reader.get_checkpoint_mark()`, marks the primary as done, and closes the current reader. The fraction parameter is accepted but ignored for proportional redistribution. Before `reader.start()` has been called, `try_split()` returns `None` because no checkpoint is available yet.

4.2.6 Watermark, Checkpoint Finalization, and Failure Semantics

The wrapper reuses Beam's existing fault-tolerance and serialization semantics. Watermark propagation uses Beam's manual watermark estimator. Monotonicity is enforced via `max(...)` with `MIN_TIMESTAMP` and `MAX_TIMESTAMP` as sentinels. A custom `WatermarkEstimatorProvider` initializes the estimator from the restriction's stored watermark on each resume, so the watermark state survives checkpoint/restore without a separate persistence path.

`CheckpointMark.finalize_checkpoint()` is the only commit-like action and runs only after a durable bundle commit via `DoFn.BundleFinalizerParam`. The finalizer is registered in the `finally` block of `process()` (see [Appendix B](#)) so it is never skipped regardless of exit path. When `try_split()` fires asynchronously, it closes the reader but caches the last checkpoint mark inside the tracker; the `finally` block retrieves this cached mark via `tracker.last_checkpoint_mark()` instead of calling the now-closed reader. Checkpoint creation is idempotent since `get_checkpoint_mark()` may be invoked again before an earlier mark has been finalized.

The wrapper uses a separate coder for each piece of state: `default_output_coder()` for output elements, `UnboundedSource.get_checkpoint_mark_coder()` for checkpoint marks, and a restriction coder that serializes the source object, checkpoint mark, and watermark. The source is pickled by default, mirroring Java’s `SerializableCoder`. Sources with non-pickleable state must implement `__getstate__`/`__setstate__` or provide a custom coder. This will be documented in the ABC docstring.

Live readers are never included in the serialized restriction. On the same worker, the reader is reused via the in-memory cache described in [§4.2.4](#). When work moves to a different worker due to failover, autoscaling, or a restriction split, the wrapper reconstructs a fresh reader from the persisted checkpoint mark.

4.3 Watch Transform (Essential)

4.3.1 API Design

The Python Watch API mirrors Java and retains explicit coder hooks when inference is insufficient:

```

1 Watch.growth_of(poll_fn)
2     .with_poll_interval(duration)
3     .with_termination_per_input(condition)
4     .with_output_key_fn(key_fn)
5     .with_output_coder(coder)
6     .with_output_key_coder(coder)

```

4.3.2 SDF Internals

Watch’s internal SDF (`WatchGrowthFn`) uses a two-subclass `GrowthState` hierarchy that mirrors Java Watch’s external semantics. `PollingGrowthState` holds the live polling state: completed-output hashes, watermark, and termination-condition state. `NonPollingGrowthState` is the split-residual form: it carries the already-updated completed set, the still-pending outputs, the current watermark, and the termination state so the next invocation can replay buffered outputs before resuming polling. The `GrowthStateTracker` mirrors Java’s `Watch.GrowthTracker` with a three-case `try_split()` path: active polling, replaying pending outputs, or already-terminal state. Full pseudocode for both the core loop and `try_split()` appears together in [Appendix C](#); in the active-polling case, the primary is marked done and the residual snapshots the updated completed set plus still-unemitted outputs.

On each invocation, the SDF polls, filters already-seen outputs, emits new outputs with their discovery timestamps, and updates state. If `PollResult` does not explicitly provide a watermark, the implementation infers one from the earliest newly observed output timestamp, matching Java Watch semantics. It then checks termination and either reschedules via `ProcessContinuation.resume()` or terminates by returning from `process()` without yielding a `ProcessContinuation`.

State updates (adding to the completed set, advancing watermark, updating termination state) are performed through the tracker’s own mutation methods (`add_completed()`,

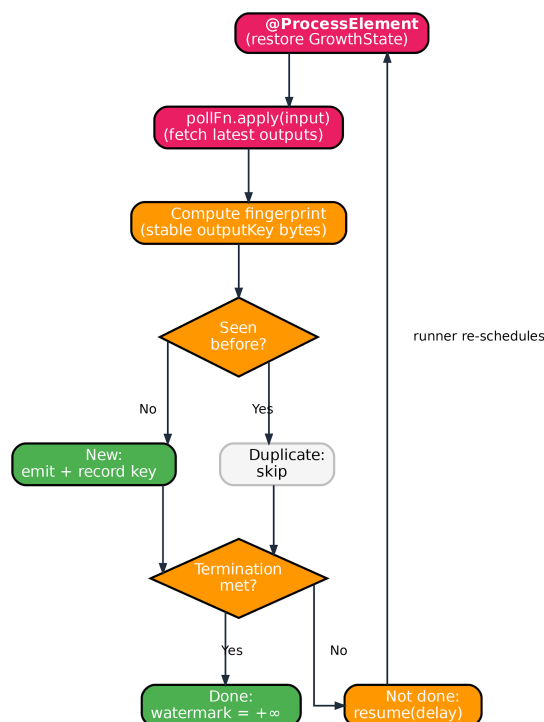


Figure 3: Watch transform: polling, deduplication, termination, and rescheduling.

`update_watermark()`, `mark_done()`) rather than by directly modifying the object returned by `current_restriction()`, since some `RestrictionTracker` implementations return a copy. The [Appendix C](#) pseudocode uses these tracker methods consistently.

`try_claim(state)` accepts the current `GrowthState` as its position argument, mirroring Java's `GrowthTracker.tryClaim(GrowthState)`. Semantically, `try_claim` transfers ownership of the state object to the tracker: the tracker stores the passed state as its internal restriction, checks `is_done`, and returns `True/False`. After a successful claim, `process()` must route all mutations through the tracker rather than through the originally passed reference.

Thread safety. The runner may call `try_split()` while `process()` is still updating the tracker. Without a lock, `try_split()` could observe a partially updated state and produce an incorrect split. A `threading.Lock` prevents that by making each update and split operation atomic with respect to each other.

4.3.3 Hash Stability: A Correctness Requirement

Python's built-in `hash()` is intentionally randomized per process (`PYTHONHASHSEED`) and is therefore not stable across workers or restarts. The Python Watch port will use a stable 16-byte hash computed from the encoded output-key bytes, using a standard-library implementation such as `hashlib.blake2b(digest_size=16)`. `blake2b` is in the standard library, so no new Beam dependency is needed. At 128 bits the collision probability remains negligible at realistic scales here; the birthday bound is around 2^{64} entries.

Because dedup hashing sits on the hot path of high-throughput streaming pipelines, the chosen implementation will be benchmarked during community bonding / early coding and kept internal to `GrowthState` so the implementation can switch to another stable option (for example `xxhash`) without API changes if profiling shows a meaningful bottleneck.

4.3.4 Memory Management and State Eviction

Completed-output state cannot grow without bound in a long-running streaming pipeline. Java's Watch currently stores the completed set without eviction, which is a known scalability limitation ([Issue #18459](#)). The Python port addresses this from the outset with bounded LRU retention: an LRU-ordered set of output-key hashes with a configurable `max_completed_keys` cap (default tuned during community bonding).

When the set exceeds `max_completed_keys`, the oldest entries are evicted and the transform may re-emit previously seen outputs. Re-emitted outputs receive a fresh discovery timestamp and may produce duplicates, but this does not violate watermark monotonicity. Potential duplicate records caused by re-emitted outputs can be handled via idempotent sinks, triggers, or downstream windowing. Users with stricter dedup requirements can increase `max_completed_keys` if memory allows. Alternatively, they can pair Watch with an idempotent sink or a downstream dedup step.

Watermark-driven eviction would allow more precise cleanup based on timestamp semantics, but requires accurate watermark signals from the poll function and is left as a future refinement. When a Watch input reaches its termination condition, all associated states are released.

GrowthState serialization. `PollingGrowthState` contains a completed-key hash set that must survive checkpoint/restore. The restriction coder encodes:

1. completed `blake2b` hashes as a length-prefixed sequence of fixed-size 16-byte digests.
2. the current watermark via `TimestampCoder`.
3. the serialized termination-condition state.
4. the `is_done` flag.

`NonPollingGrowthState` encodes its pending-output list using the output element's coder plus `TimestampCoder` for the associated watermark. The coders follow Beam's existing restriction-coder patterns and will be validated via round-trip serialization tests.

4.3.5 Termination Conditions

Termination is composable. Table 3 summarizes the six conditions that the Python Watch port will expose.

Condition	Behavior
<code>never()</code>	Poll until <code>PollResult</code> reports completion
<code>after_total_of(duration)</code>	Stop after fixed elapsed time
<code>after_time_since_new_output(duration)</code>	Stop after a period with no new output
<code>after_iterations(n)</code>	Stop after a fixed number of poll iterations; useful for deterministic tests
<code>either_of(c1, c2)</code>	Boolean OR combinator
<code>all_of(c1, c2)</code>	Boolean AND combinator

Table 3: Watch termination conditions.

4.4 Comprehensive Test Suite (Essential)

All new public classes and methods will have test coverage for expected behavior and edge cases. The test suite validates:

1. Watermark monotonicity, late-data paths, and watermark regression prevention.
2. Checkpoint recovery across simulated failures.

3. Polling termination on success, timeout, and empty paths.
4. Deduplication correctness with record-ID contract verification.

Serialization-compatibility validation covers coder round trips for all contract-bearing states across `_SDFUnboundedSourceRestrictionTracker`, `GrowthState`, and `CheckpointMark`.

`TestStream`-based scenarios exercise controlled watermark advancement and out-of-order data. Integration tests run on `DirectRunner` and at least one distributed runner, with `Dataflow` prioritized because checkpoint behavior differences are most likely to surface there; `Flink` coverage will be pursued as an additional validation path when practical.

Serialization verification for distributed execution includes cross-version safety documentation. A short CI-friendly stress-test variant (about 5–10 minutes) validates the same core invariants at smaller scale, while a longer soak run is used for manual pre-submission validation or as a follow-up post-commit job if appropriate for Beam infrastructure.

4.5 User-Facing Documentation (Essential)

Documentation deliverables include SDK docstrings for all new public classes and methods, a programming guide section on authoring custom streaming IOs with `UnboundedSource`, usage examples for `Watch` (file discovery, API polling), migration guidance for users currently implementing raw SDF for these patterns, and guidance on using custom Coders for `CheckpointMark` and source serialization in production deployments with coder registration.

4.6 Optional / Stretch

- Update Python `fileio.MatchContinuously` to use the new `Watch` transform internally.
- Implement a native Python streaming IO connector using the new `UnboundedSource` API as an end-to-end validation of the abstraction.

5. Proof of Concept and Early Validation

The design above is grounded in standalone PoCs built before the coding period ([PoC repository](#)).

The goal was to validate runtime semantics instead of only sketching APIs: checkpointing, watermark progression, stop-after-failed-claim behavior, deferred resumption, and `Watch` polling and termination flow.

PoC Area	What Is Already Validated	Status
UnboundedSource wrapper	Restriction lifecycle, pause/resume, checkpoint round trip, stop-after-failed-claim behavior, and checkpoint-safe <code>try_split()</code> with pre-claim guard.	77 tests
Watch transform	Polling loop structure, per-input <code>GrowthState</code> modeling, dedup skeleton, and rescheduling / termination control flow.	60 tests
Design readiness	Prototype code and diagrams already exist, so implementation starts from working code.	137 tests

Table 4: PoC validation summary.

6. Essential Deliverables

Each deliverable targets a merged pull request to the Apache Beam repository. To manage merge timing, draft PRs will be opened early and kept small. If review cycles extend beyond the coding period, deliverables will be at minimum code-reviewed, tests-passing, and PR-approved.

D1 – Python UnboundedSource API + SDF-backed runtime wrapper

A functional wrapper API for UnboundedSource and UnboundedReader built on Splittable DoFn. ABCs in the Python SDK; checkpoint save/restore and watermark reporting verified through the SDF wrapper; one end-to-end example.

D2 – Python Watch Transform

A Python counterpart to the Java Watch API for periodic polling. Periodic polling with user-supplied PollFn; all six termination conditions; watermark monotonicity; graceful no-data polling.

D3 – Comprehensive Test Suite

Unit and integration tests for all public API classes and methods, specifically covering watermarks, checkpointing, and polling termination conditions, plus serialization verification, TestStream scenarios, and DirectRunner integration as specified in Section 4.4. Tests stay even if the schedule slips.

D4 – User-Facing Documentation

Docstrings, programming guide section, usage examples, and migration guidance as specified in Section 4.5.

Estimated effort for the ~350-hour large-project budget:

- **D1** (~135h): UnboundedSource API + wrapper core. If D1 overruns, freeze convenience APIs and protect test coverage and invariants first.
- **D2** (~105h): Watch core. If D2 overruns, defer non-essential polish and drop `fileio.MatchContinuously` work.
- **D3** (~70h): Test suite + cross-runner validation. Tests are non-negotiable. Under time pressure, manual stress tests and optional runner coverage can be compressed.
- **D4** (~40h): Docstrings, guide section, examples, and migration notes. API docs and migration guidance are protected.

7. Results for the Apache Beam Community

Python engineers can write custom streaming connectors (Pub/Sub-like message sources, CDC readers) using familiar abstract classes without wiring SDF internals manually. Watch transform provides a polling primitive for file discovery, API monitoring, and change detection.

For maintainers, this addresses Issues [#19137](#) and [#21521](#), with a regression suite protecting future SDF changes. Longer term, the wrapper improves Java/Python parity for streaming IO authoring, and Watch can underpin future polling-based transforms such as `fileio.MatchContinuously`.

8. Schedule (12 Weeks; May 25 – August 16, 2026)

Priority order under scope pressure: UnboundedSource wrapper, then Watch, then test coverage, then documentation, then stretch work.

Week / Dates	Explicit Weekly Exit Criteria
Bonding (May 1–24)	API necessity and design shape are treated as community-validated items; bonding begins with a design document and dev-list discussion before implementation commits. Watch design outline started; PoC extended for watermark, dedup, and error-handling notes; review cadence confirmed.
W1 (May 25–31)	ABCs drafted; checkpoint coder interface defined; pause/resume and harness-triggered checkpoint note written. [D1]
W2 (Jun 1–7)	Options plumbing, <code>DONE_RESTRICTION</code> semantics, and <code>_SDFUnboundedSourceRestrictionTracker</code> implemented; any spillover moves to W6. [D1]
W3 (Jun 8–14)	WatermarkEstimator wiring, record-ID dedup path, and <code>Read(UnboundedSource)</code> expansion implemented end-to-end; first draft PR for D1 opened for early mentor review. [D1]
W4 (Jun 15–21)	D3 tests for D1: unit tests for checkpoint round trip, watermark monotonicity, dedup contract, checkpoint-safe <code>try_split()</code> pre-claim guard, zero-fraction checkpoint semantics, and serialization-compatibility validation. [D1/D3]
W5 (Jun 22–28)	D3 integration tests: <code>TestStream</code> and <code>DirectRunner</code> integration added; mock <code>CountingUnboundedSource</code> runs end-to-end; first draft PR feedback incorporated and follow-up revisions posted. [D1/D3]
W6 (Jun 29–Jul 5)	Buffer week for W2–W5 spillover; D1 review iteration continues toward approval/merge target; Watch design doc circulated for mentor and committer feedback. [D1/D2]
*W7 (Jul 6–12)	Midterm target: UnboundedSource wrapper implementation complete, tested, draft PR opened, and first review round substantially incorporated; Watch API refined from mentor feedback. [D1]
W8 (Jul 13–19)	Watch API frozen: <code>PollFn</code> , <code>PollResult</code> , <code>TerminationCondition</code> , hashing choice, eviction guidance, and retry semantics; first draft PR for D2 opened once API shape stabilizes. [D2]
W9 (Jul 20–26)	<code>Watch.growth_of()</code> , <code>WatchGrowthFn</code> , and <code>GrowthState</code> hierarchy implemented with stable dedup keys and optional watermark support; initial D2 review feedback addressed. [D2]
W10 (Jul 27–Aug 2)	D3 Watch tests complete: polling cadence, dedup across rounds, termination paths, eviction boundaries, serialization-compatibility validation, and distributed-runner validation, prioritizing <code>Dataflow</code> where practical. Full D3 test suite finalized across D1 and D2; D2 review iteration continues toward approval/merge target. [D2/D3]
W11 (Aug 3–9)	D4 complete: user docs, guide section, examples, migration notes, and docstrings finalized; final packaging and evaluation after core completion; stretch only if essentials are stable and core PRs are already in late review. [D4]
W12 (Aug 10–16)	W12 reserved for final D3/D4 polish, packaging, evaluation, and only optional stretch/demo work (<code>fileio.MatchContinuously</code>) if all essentials are already stable; any remaining mentor feedback on core PRs is addressed here as needed. [D3/D4/Stretch]

Table 5: Week-by-week plan with explicit exit criteria.

Buffers: W6 absorbs early wrapper spillover. W11–W12 reserve time for D4 instead of compressing docs into the final days.

9. Other Commitments

GSoC will be my primary commitment during the coding period, with no internship, job, or other recurring obligation in parallel. I will contribute about 30 hours per week throughout the coding period. I only have one short university essay due in early June. It is a minor commitment and already accounted for in the schedule.

10. Community Engagement

Mentor collaboration: The project scope and design direction were discussed with mentor Yi Hu during the pre-proposal period. The proposal itself was refined by multiple rounds of his feedback, including structural changes (moving pseudocode to appendices), API naming alignment with Python SDK conventions, and additional runner-compatibility considerations.

Design discussion: I initiated a [design discussion thread](#) on `dev@beam.apache.org` outlining the approach for bringing UnboundedSource and Watch to the Python SDK.

Proof of concept: Before the coding period, I validated the core mechanics that matter most here — UnboundedSource lifecycle, checkpoint and watermark flow, deferred resumption, and Watch polling and termination behavior.

Merged PRs: Working through PRs [#37565](#), [#37428](#), [#37355](#), and [#37299](#) gets me familiar with Beam's contribution workflow, CI system, and code review process.

Issue context: I have studied Issues [#19137](#) and [#21521](#) in depth, including linked discussion.

Communication plan: During the project I will stay in close contact with my mentor through weekly calls, regular check-ins, mailing-list updates, and early draft PRs. For larger design changes, I will circulate short design notes before implementation and ask for review from the community. I will turn review feedback around quickly in small, focused iterations.

Design validation: Given prior maintainer feedback, API necessity and final design shape are treated as validation items rather than fixed assumptions. The project begins with a design document and community discussion before committing to the implementation path.

11. Risks and Mitigations

Risk	Mitigation
RestrictionTracker state machine complexity	Front-load to W2; PoC already validates pause/resume; use W6 as explicit spillover buffer.
Runner-specific checkpoint behavior	Validate on <code>DirectRunner</code> and <code>Dataflow</code> ; test under high-frequency checkpointing to confirm no data loss; investigate and document runner-specific SDF limitations (e.g., incomplete self-checkpoint or dynamic-split support); correctness does not depend on <code>resume_delay</code> fidelity.
Stretch-goal scope creep	Gate stretch work behind all essential deliverables; drop it first if behind schedule.
Review bandwidth delays merge	Open draft PRs early; keep PRs small and documented; aim for merge within the coding period; if review cycles extend, deliverables are at minimum code-reviewed and PR-approved.
Late-stage schedule compression	Surface D4 in W11–W12; keep W6 as the main technical buffer before any core deliverable is compromised.

Table 6: Risk matrix.

12. References

1. [GitHub Issue #19137](#) – Support for custom Python `UnboundedSource`
2. [GitHub Issue #21521](#) – Python Watch transform
3. [GitHub Issue #18459](#) – Improve scalability of the Watch transform
4. [Java Watch.java](#) – `sdk/java/core/.../transforms/Watch.java`
5. [Java UnboundedSource.java](#) – `sdk/java/core/.../io/UnboundedSource.java`
6. [Beam Python iobase.py](#) – `sdk/python/apache_beam/io/iobase.py`
7. [PR #37278](#) – [Docs] Fix grammar in streaming configuration comments
8. [PR #37299](#) – [Python] Fix `AttributeError` in `ExternalTransform.expand`
9. [PR #37355](#) – [Java] Fix `DoFnInvoker` cache collisions for generic types
10. [PR #37428](#) – [Python] Add content-aware dynamic batching to `RunInference`
11. [PR #37532](#) – [Python] Stateless bundle-local size-aware batching for `RunInference`
12. [PR #37565](#) – [Python] Length-aware keying to minimize padding in `BatchElements`
13. [PR #37945](#) – [Python] Expose length-aware batching in all `ModelHandler` subclasses
14. [PR #37965](#) – [Python] Expand SDF by default in `PortableRunner`
15. [GSoC PoC Repository](#) – `Eliaaazzz/beam-native-streaming-poc`

A. UnboundedSource ABC Definitions

```

1 class CheckpointMark(abc.ABC):
2     @abc.abstractmethod
3     def finalize_checkpoint(self): ...
4     def get_offset_limit(self):
5         raise NotImplementedError # required if offset-based dedup
6
7 class UnboundedReader(abc.ABC):
8     # Abstract: start, advance, get_current, get_current_timestamp,
9     #             get_watermark, get_checkpoint_mark, close
10    def get_current_record_id(self):
11        raise NotImplementedError # optional
12    def get_current_record_offset(self):
13        raise NotImplementedError # required if offset-based dedup
14    def get_split_backlog_bytes(self):
15        return BACKLOG_UNKNOWN # optional autoscaling hint
16    def get_total_backlog_bytes(self):
17        return BACKLOG_UNKNOWN # optional autoscaling hint
18
19 class UnboundedSource(abc.ABC):
20     # Abstract: split, create_reader, get_checkpoint_mark_coder
21     def offset_based_deduplication_supported(self):
22         return False
23     def requires_deduping(self):
24         return False
25
26 # --- SDF Restriction (follows _SDFBoundedSourceRestriction pattern
27 #     in iobase.py) ---
28
29 @dataclass
30 class _SDFUnboundedSourceRestriction:
31     source: UnboundedSource
32     checkpoint_mark: Optional[CheckpointMark] = None
33     watermark: Timestamp = MIN_TIMESTAMP
34     is_done: bool = False
35
36
37 # --- Per-element value passed from tracker to process() ---
38
39 UnboundedSourceValue = NamedTuple(
40     'UnboundedSourceValue',
41     [('value', Any),
42      ('timestamp', Timestamp),
43      ('watermark', Timestamp),
44      ('record_id', Optional[bytes])])
45
46
47 # --- Restriction Coder
48 #     (follows _SDFBoundedSourceWrapperRestrictionCoder pattern) ---
49
50 class _SDFUnboundedSourceWrapperRestrictionCoder(Coder):
51     def __init__(self, checkpoint_mark_coder):
52         self._ckpt_coder = checkpoint_mark_coder
53
54     def encode(self, restriction, out):
55         # source_coder(source) + ckpt_coder(mark) + TimestampCoder(wm)
56         ...
57
58     def decode(self, inp):
59         # inverse; reconstruct _SDFUnboundedSourceRestriction
60         ...

```

B. UnboundedSource SDF Pseudocode

```

1 class _SDFUnboundedSourceRestrictionTracker(RestrictionTracker):
2     # STATE 1: READING      - poll/read attempt claimed; may emit 0..1 records
3     # STATE 2: WAITING      - no data right now; defer remainder + continuation
4     # STATE 3: CHECKPOINTING - runner/self requested checkpoint; stop claiming
5
6     def try_claim(self, position):
7         # position: None (gating claim; tracker only checks is_done)
8         if self._current_restriction.is_done:
9             return False
10        self._last_claimed = position
11        return True
12
13    def try_split(self, fraction_of_remainder):
14        # Any fraction triggers checkpoint-cut; pre-start returns None.
15        if self._current_restriction.is_done or not self._started:
16            return None
17        checkpoint = self._reader.get_checkpoint_mark()
18        # Cache mark so process() can register finalization after close.
19        self._last_checkpoint_mark = checkpoint
20        residual = Restriction(
21            source, checkpoint, watermark=max(prev, cur))
22        self._current_restriction = DONE_RESTRICTION
23        self._close_reader()
24        return (DONE_RESTRICTION, residual)
25
26    # --- SDF Wrapper DoFn ---
27    def process(self, element, tracker, wm_estimator, bundle_finalizer):
28        restriction = tracker.current_restriction()
29        reader = restriction.source.create_reader(
30            self._options, restriction.checkpoint_mark)
31        try:
32            has = reader.start()
33            while True:
34                if not tracker.try_claim(None):
35                    break
36                if not has: # no data right now
37                    yield ProcessContinuation.resume(delay=poll_interval)
38                    return
39                value = UnboundedSourceValue(reader.get_current(), ...)
40                wm_estimator.set_watermark(max(prev_wm, value.watermark))
41                yield value.value
42                emitted += 1
43                if emitted >= max_per_invocation:
44                    yield ProcessContinuation.resume()
45                    return
46            has = reader.advance()
47        finally:
48            # Retrieve checkpoint mark from tracker cache (set by try_split)
49            # or directly from reader if try_split hasn't fired.
50            ckpt = (tracker.last_checkpoint_mark()
51                  or reader.get_checkpoint_mark())
52            bundle_finalizer.after_bundle_commit(
53                lambda: ckpt.finalize_checkpoint())
54            reader.close()
55
56    class _SDFUnboundedSourceRestrictionProvider(RestrictionProvider):
57        def initial_restriction(self, source):
58            return Restriction(source, checkpoint_mark=None, is_done=False)
59        def split(self, source, restriction):
60            for sub_source in source.split(desired_num_splits, options):
61                yield Restriction(sub_source, checkpoint_mark=None)
62        def restriction_coder(self):
63            return _SDFUnboundedSourceWrapperRestrictionCoder(
64                self._checkpoint_mark_coder)

```

C. Watch SDF Core Loop

```

1 @dataclass
2 class PollingGrowthState:
3     completed: Set[bytes]; watermark: Timestamp
4     termination_state: Any; is_done: bool = False
5 @dataclass
6 class NonPollingGrowthState:
7     completed: Set[bytes]; pending: List[OutputT]
8     watermark: Timestamp; termination_state: Any
9     is_done: bool = False
10
11 class GrowthStateTracker(RestrictionTracker):
12     # All state access guarded by self._lock (runner may call
13     # try_split from a different thread during active polling).
14     def try_claim(self, state):
15         with self._lock:
16             if self._current_restriction.is_done: return False
17             self._current_restriction = state # ownership transfer
18             return True
19     def add_completed(self, key_hash): ...
20     def update_watermark(self, wm): ...
21     def mark_done(self): ...
22     def set_pending_outputs(self, outputs): ...
23     def mark_output_emitted(self): ...
24     def pop_pending_output(self): ...
25     def remaining_pending_outputs(self): ...
26     def resume_polling(self): ...
27     def try_split(self, fraction_of_remainder):
28         with self._lock:
29             state = self._current_restriction
30             if state.is_done: return None
31             # Case: replaying pending outputs (NonPollingGrowthState)
32             if isinstance(state, NonPollingGrowthState):
33                 idx = max(1, int(len(state.pending) * fraction_of_remainder))
34                 if idx >= len(state.pending): return None
35                 primary = NonPollingGrowthState(
36                     set(state.completed), state.pending[:idx],
37                     state.watermark, state.termination_state)
38                 residual = NonPollingGrowthState(
39                     set(state.completed), state.pending[idx:],
40                     state.watermark, state.termination_state)
41                 self._current_restriction = primary
42                 return (primary, residual)
43             # Case: active polling -- mark done, package unemitted as residual
44             state.is_done = True
45             residual = NonPollingGrowthState(
46                 set(state.completed), self.remaining_pending_outputs(),
47                 state.watermark, state.termination_state)
48             return (state, residual)
49     def process(self, element, tracker, wm_estimator):
50         state = tracker.current_restriction()
51         if not tracker.try_claim(state): return
52         # Replay path: emit pending outputs from a prior split
53         if isinstance(state, NonPollingGrowthState):
54             while True:
55                 output = tracker.pop_pending_output()
56                 if output is None: break
57                 yield windowed_value((element, output.value), timestamp=output.timestamp)
58                 tracker.resume_polling()
59                 yield ProcessContinuation.resume(delay=self._poll_interval)
60             return
61         # Polling path: invoke user PollFn, dedup, emit new outputs
62         poll_result = self._poll_fn(element)
63         completed = tracker.current_restriction().completed
64         new_outputs = [o for o in poll_result.outputs if _key_hash(o.value) not in completed]
65         tracker.set_pending_outputs(new_outputs)
66         for output in new_outputs:
67             tracker.add_completed(_key_hash(output.value))
68             tracker.mark_output_emitted()
69             yield windowed_value((element, output.value), timestamp=output.timestamp)
70         inferred_wm = poll_result.watermark or min((o.timestamp for o in new_outputs), default=None)
71         if inferred_wm is not None:
72             tracker.update_watermark(inferred_wm)
73         wm_estimator.set_watermark(tracker.current_restriction().watermark)
74         if self._termination.should_stop(tracker.current_restriction(), new_outputs):
75             tracker.mark_done(); return
76         yield ProcessContinuation.resume(delay=self._poll_interval)

```